Application de Filtres d'Image

avec OpenCV et Python

Rapport de Projet

Traitement d'Image / Vision par Ordinateur

Groupe : TPE2

Membres : Malak BOUKRIM, Oussama BOUFFI , Oumayma JEBRANE , Hiba LAHRACH

Encadrant : [Ilias TOUGUI]

Établissement : [UNIVERSITE INTERNATIONAL DE RABAT]

Date : 21 avril 2026

Résumé

Ce rapport présente la conception et l'implémentation d'un notebook interactif de traitement d'image, développé en Python dans un environnement Google Colab. En utilisant la bibliothèque OpenCV, le notebook permet à l'utilisateur de charger une image, de sélectionner un filtre via une interface `ipywidgets`, et de visualiser le résultat directement dans le dashboard de Colab sous forme de comparaison côte à côte. Dix filtres ont été implémentés, couvrant le lissage, la netteté, la détection de relief, la transformation colorimétrique et des effets artistiques. Les résultats s'affichent instantanément dans les cellules de sortie du notebook, sans nécessiter d'interface graphique externe. Chaque filtre atteint l'effet visuel attendu, avec des comportements distincts selon sa catégorie.

Table des matières

1 Introduction **3**

2 Présentation du projet **3**
2.1 Contexte et objectifs . . . 3
2.2 Environnement technique . . . 3
2.3 Filtres implémentés . . . 3

3 Conception **4**
3.1 Architecture générale . . . 4
3.2 Gestion de l'état . . . 4
3.3 Gestion des cas particuliers . . . 4

4 Implémentation **4**
4.1 Filtres linéaires . . . 4
4.1.1 Filtre Blur (lissage) . . . 4
4.1.2 Filtre Sharpen (netteté) . . . 4
4.1.3 Filtre Emboss (relief) . . . 5
4.2 Filtres non linéaires . . . 5
4.2.1 Filtre Median . . . 5
4.2.2 Filtre Bilateral . . . 5
4.3 Transformations couleur et effets artistiques . . . 6
4.3.1 Filtre Grayscale . . . 6
4.3.2 Filtre Sepia . . . 6
4.3.3 Filtre Pencil Sketch . . . 6
4.3.4 Filtre Vignette . . . 6
4.3.5 Filtre Blend . . . 7
4.4 Interface interactive . . . 7

5 Résultats **7**
5.1 Comportement observé par catégorie . . . 7
5.2 Comparaison synthétique . . . 8
5.3 Limitations observées . . . 8

6 Conclusion **8**

1. Introduction

Le traitement d'image est un domaine fondamental de la vision par ordinateur et du traitement du signal. Il consiste à manipuler les pixels d'une image numérique afin d'en modifier l'apparence ou d'en extraire des informations utiles. Parmi les opérations les plus courantes, les *filtres d'image* permettent de lisser le bruit, d'accentuer les contours, de modifier les couleurs ou de produire des effets artistiques.

Ce projet s'inscrit dans le cadre d'un cours de traitement d'image. Son objectif est de mettre en pratique les notions de convolution, de filtrage linéaire et non linéaire, ainsi que de transformation colorimétrique, en les intégrant dans une application interactive utilisable directement dans un notebook Python.

Ce rapport est organisé comme suit : la section 2 présente le projet dans son ensemble ; la section 3 décrit les choix de conception ; la section 4 détaille l'implémentation des différents composants ; la section 5 analyse les résultats obtenus ; enfin, la section 6 conclut et ouvre des perspectives d'amélioration.

2. Présentation du projet

2.1. Contexte et objectifs

L'objectif principal de ce projet est de développer une application permettant d'appliquer des filtres d'image de manière interactive. L'utilisateur doit pouvoir :

- charger une image depuis son ordinateur,
- sélectionner un filtre dans une liste déroulante,
- visualiser le résultat avant/après en temps réel,
- sauvegarder l'image filtrée.

2.2. Environnement technique

L'application est développée en Python et s'exécute dans un notebook Google Colab. Les bibliothèques utilisées sont les suivantes :

Table 1 – Bibliothèques utilisées dans le projet.

Bibliothèque	**Version**	**Rôle**
OpenCV (`cv2`)	$\geq$ 4.x	Traitement d'image
NumPy	$\geq$ 1.x	Calcul matriciel
Matplotlib	$\geq$ 3.x	Affichage des images
`ipywidgets`	$\geq$ 7.x	Interface interactive
`google.colab.files`	—	Upload/download de fichiers

2.3. Filtres implémentés

Dix filtres ont été développés, répartis en trois catégories :

- **Filtres linéaires :** Blur, Sharpen, Emboss.
- **Filtres non linéaires :** Median, Bilateral.
- **Transformations couleur et effets artistiques :** Grayscale, Sepia, Pencil Sketch, Vignette, Blend.

3. Conception

3.1. Architecture générale

L'application est structurée autour de trois composants distincts :

1. **Couche de traitement :** un ensemble de fonctions Python indépendantes, chacune implémentant un filtre. Chaque fonction prend une image RGB en entrée et retourne une image transformée.
2. **Couche de registre :** un dictionnaire Python associant le nom affiché de chaque filtre à sa fonction correspondante.
3. **Couche d'interface :** des widgets `ipywidgets` (menu déroulant, boutons) connectés aux fonctions via des gestionnaires d'événements.

Cette séparation permet d'ajouter un nouveau filtre en deux étapes seulement : écrire la fonction, puis l'enregistrer dans le dictionnaire.

3.2. Gestion de l'état

L'application maintient deux variables globales :
— `image` : l'image originale chargée par l'utilisateur (jamais modifiée).
— `current_result` : le résultat du dernier filtre appliqué (utilisé lors de la sauvegarde).
Cette distinction garantit que l'image de référence reste intacte pour toutes les comparaisons visuelles.

3.3. Gestion des cas particuliers

Certains filtres retournent une image en niveaux de gris (un seul canal), tandis que d'autres retournent une image RGB (trois canaux). La fonction `show_comparison` et la fonction `save_result` traitent ces deux cas explicitement pour éviter toute erreur d'affichage ou de sauvegarde.

4. Implémentation

4.1. Filtres linéaires

Les filtres linéaires reposent sur l'opération de *convolution* : chaque pixel de sortie est une combinaison linéaire de ses voisins dans l'image d'entrée, pondérée par un noyau (*kernel*).

4.1.1 Filtre Blur (lissage)

Ce filtre applique une moyenne uniforme sur un voisinage 7×7. Il atténue les hautes fréquences (bruit, détails fins) et constitue un filtre passe-bas classique.

```python
def blur_filter(img):
    return cv2.blur(img, (7, 7))
```

Listing 1 – Filtre Blur par convolution uniforme.

4.1.2 Filtre Sharpen (netteté)

Le noyau de netteté amplifie le pixel central tout en soustrayant ses voisins immédiats. Cela renforce les transitions de haute fréquence, produisant un effet proche d'un filtre passe-haut.

```python
def sharpen_filter(img):
    kernel = np.array([
        [ 0, -1,  0],
        [-1,  5, -1],
        [ 0, -1,  0]
    ])
    return cv2.filter2D(img, -1, kernel)
```

Listing 2 – Filtre Sharpen avec noyau laplacien.

Le noyau utilisé vérifie bien $\sum k_{ij} = 1$, ce qui préserve la luminosité globale de l'image.

4.1.3 Filtre Emboss (relief)

Le noyau d'emboss est asymétrique : il calcule une différence directionnelle entre pixels opposés, simulant un éclairage rasant qui crée un effet tridimensionnel. Un décalage de +128 est ajouté pour centrer le résultat dans la plage $[0, 255]$.

```python
def emboss_filter(img):
    kernel = np.array([
        [-2, -1, 0],
        [-1,  1, 1],
        [ 0,  1, 2]
    ])
    embossed = cv2.filter2D(img, -1, kernel)
    return np.clip(embossed + 128, 0, 255).astype(np.uint8)
```

Listing 3 – Filtre Emboss avec décalage de niveaux.

4.2. Filtres non linéaires

Contrairement aux filtres linéaires, les filtres non linéaires ne peuvent pas s'exprimer sous forme de convolution.

4.2.1 Filtre Median

Chaque pixel est remplacé par la *médiane* de ses voisins dans une fenêtre 5×5. Ce filtre est particulièrement efficace contre le bruit impulsionnel (sel et poivre) car la médiane est robuste aux valeurs aberrantes.

```python
def median_filter(img):
    return cv2.medianBlur(img, 5)
```

Listing 4 – Filtre Median.

4.2.2 Filtre Bilateral

Le filtre bilatéral effectue un lissage tout en préservant les contours. Il combine un filtre gaussien spatial et un filtre gaussien en intensité : seuls les pixels à la fois proches spatialement *et* similaires en couleur contribuent au lissage.

```python
def bilateral_filter(img):
    return cv2.bilateralFilter(img, 9, 75, 75)
```

Listing 5 – Filtre Bilateral preservant les contours.

Les paramètres `sigmaColor=75` et `sigmaSpace=75` contrôlent respectivement la plage d'intensité et la plage spatiale du filtre.

4.3. Transformations couleur et effets artistiques

4.3.1 Filtre Grayscale

Conversion de l'espace couleur RGB vers les niveaux de gris. OpenCV utilise la formule de luminance perceptuelle :

$$Y = 0.299\,R + 0.587\,G + 0.114\,B$$

```python
def grayscale_filter(img):
    return cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
```

Listing 6 – Conversion en niveaux de gris.

4.3.2 Filtre Sepia

La transformation sépia applique une matrice linéaire 3×3 sur les canaux RGB pour simuler le rendu photographique vintage :

$$\begin{pmatrix} R' \\ G' \\ B' \end{pmatrix} = \begin{pmatrix} 0.393 & 0.769 & 0.189 \\ 0.349 & 0.686 & 0.168 \\ 0.272 & 0.534 & 0.131 \end{pmatrix} \begin{pmatrix} R \\ G \\ B \end{pmatrix}$$

```python
def sepia_filter(img):
    kernel = np.array([
        [0.393, 0.769, 0.189],
        [0.349, 0.686, 0.168],
        [0.272, 0.534, 0.131]
    ])
    sepia = cv2.transform(img, kernel)
    return np.clip(sepia, 0, 255).astype(np.uint8)
```

Listing 7 – Filtre Sepia par transformation matricielle.

4.3.3 Filtre Pencil Sketch

Cet effet imite un dessin au crayon en quatre étapes : (1) conversion en niveaux de gris, (2) inversion des niveaux, (3) flou gaussien sur l'image inversée, (4) division pixel par pixel entre le gris original et l'image floutée inversée.

```python
def pencil_sketch_filter(img):
    gray     = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
    inverted = 255 - gray
    blurred  = cv2.GaussianBlur(inverted, (21, 21), 0)
    sketch   = cv2.divide(gray, 255 - blurred, scale=256)
    return sketch
```

Listing 8 – Effet dessin au crayon.

4.3.4 Filtre Vignette

Un masque gaussien 2D est construit en faisant le produit externe de deux noyaux gaussiens 1D (un horizontal, un vertical). Ce masque, normalisé entre 0 et 1, est multiplié par chaque canal de l'image pour assombrir progressivement les bords.

```python
def vignette_filter(img):
    rows, cols = img.shape[:2]
    kernel_x = cv2.getGaussianKernel(cols, cols / 2)
    kernel_y = cv2.getGaussianKernel(rows, rows / 2)
```

```
    mask = (kernel_y @ kernel_x.T) / (kernel_y @ kernel_x.T).max()
    output = img.astype(np.float32)
    for i in range(3):
        output[:, :, i] *= mask
    return np.clip(output, 0, 255).astype(np.uint8)
```

Listing 9 – Effet vignette par masque gaussien.

4.3.5 Filtre Blend

Un mélange linéaire (*alpha blending*) entre l'image originale et une version floutée : $I' = 0.7 \cdot I + 0.3 \cdot \text{Blur}(I)$.

```
def blend_filter(img):
    blurred = cv2.GaussianBlur(img, (15, 15), 0)
    return cv2.addWeighted(img, 0.7, blurred, 0.3, 0)
```

Listing 10 – Effet de melange (blend).

4.4. Interface interactive

L'interface est construite avec `ipywidgets`. Un dictionnaire central relie chaque nom de filtre à sa fonction :

```
filters = {
    "Blur": blur_filter,  "Sharpen": sharpen_filter,
    "Emboss": emboss_filter, "Median": median_filter,
    "Bilateral": bilateral_filter, "Grayscale": grayscale_filter,
    "Sepia": sepia_filter, "Pencil Sketch": pencil_sketch_filter,
    "Vignette": vignette_filter, "Blend": blend_filter
}

dropdown  = widgets.Dropdown(options=list(filters.keys()))
apply_btn = widgets.Button(description="Apply Filter")
save_btn  = widgets.Button(description="Save Result")

def apply_filter(b):
    global current_result
    current_result = filters[dropdown.value](image)
    show_comparison(image, current_result, dropdown.value)

apply_btn.on_click(apply_filter)
```

Listing 11 – Registre des filtres et interface widgets.

5. Résultats

5.1. Comportement observé par catégorie

Filtres linéaires. Le filtre *Blur* produit un lissage uniforme visible dès un noyau 7×7 ; les détails fins (textures, grain) disparaissent. Le filtre *Sharpen* accentue les contours de façon notable, mais peut amplifier le bruit si l'image n'a pas été préalablement lissée. Le filtre *Emboss* donne un rendu en relief convainquant, surtout sur les zones à fort gradient (bords d'objets).

Filtres non linéaires. Le *Median* supprime efficacement les pixels isolés (bruit impulsionnel) tout en conservant les bords mieux que le Blur. Le *Bilateral* offre le meilleur compromis lissage/préservation des contours, au prix d'un temps de calcul plus élevé.

Effets artistiques. Le *Pencil Sketch* produit un effet convaincant sur les images à fort contraste. Le *Sepia* et la *Vignette* donnent un rendu photographique vintage cohérent. Le *Blend* est plus subtil et son effet est difficile à distinguer sur des images déjà lisses.

5.2. Comparaison synthétique

Table 2 – Résumé des filtres implémentés et de leurs effets.

Filtre	Type	Effet principal	Canaux en sortie
Blur	Linéaire	Lissage	RGB
Sharpen	Linéaire	Netteté / contours	RGB
Emboss	Linéaire	Relief 3D	RGB
Median	Non linéaire	Débruitage	RGB
Bilateral	Non linéaire	Lissage sélectif	RGB
Grayscale	Colorimétrique	Niveaux de gris	1 canal
Sepia	Colorimétrique	Vintage	RGB
Pencil Sketch	Artistique	Dessin au crayon	1 canal
Vignette	Artistique	Assombrissement des bords	RGB
Blend	Artistique	Douceur globale	RGB

5.3. Limitations observées

- Les filtres *Grayscale* et *Pencil Sketch* retournent une image à un seul canal, ce qui nécessite une gestion spécifique lors de la sauvegarde.
- Le filtre *Bilateral* est significativement plus lent que les autres sur des images de grande résolution.
- L'effet *Blend* est peu perceptible sur des images déjà uniformes ou peu contrastées.

6. Conclusion

Ce projet a permis de concevoir et d'implémenter une application complète de filtres d'image en Python, couvrant les grandes familles du traitement d'image numérique : filtrage linéaire par convolution, filtrage non linéaire, transformation colorimétrique et effets artistiques.

L'architecture modulaire adoptée — une fonction par filtre, un dictionnaire central comme registre — rend l'application facile à étendre. L'interface `ipywidgets` offre une expérience interactive satisfaisante dans un environnement notebook.

Parmi les perspectives d'amélioration envisageables :

- ajouter des paramètres ajustables par l'utilisateur (taille du noyau, intensité de l'effet) via des curseurs (*sliders*),
- permettre l'application de plusieurs filtres en cascade,
- ajouter un historique avec fonction d'annulation (*undo*),
- étendre l'application au traitement vidéo frame par frame, en s'appuyant sur les mêmes fonctions de filtrage.

Remerciements

Remerciements. Ce projet a été réalisé dans le cadre du cours de Traitement d'Image. Je tiens à remercier [Nom de l'encadrant] pour ses conseils et son suivi tout au long de ce travail.

Références

[1] Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.

[2] Gonzalez, R.C. & Woods, R.E. (2018). *Digital Image Processing* (4th ed.). Pearson.

[3] Harris, C.R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.

[4] Tomasi, C. & Manduchi, R. (1998). Bilateral filtering for gray and color images. *Proceedings of ICCV*, 839–846.